(19)中华人民共和国国家知识产权局

(12)发明专利申请

(10)申请公布号 CN 108205464 A
(43)申请公布日 2018.06.26

(21)申请号 201611183503.2

(22)申请日 2016.12.20

(71)申请人 阿里巴巴集团控股有限公司
地址 英属开曼群岛大开曼资本大厦一座四层847号邮箱

(72)发明人 范孝剑 张广舟 林晓斌 周正中

(74)专利代理机构 北京国昊天诚知识产权代理有限公司 11315
代理人 黄熊

(51)Int.Cl.
*G06F 9/46*(2006.01)
*G06F 9/52*(2006.01)
*G06F 17/30*(2006.01)

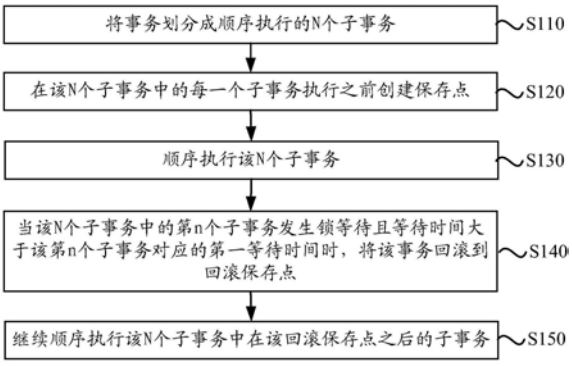
权利要求书2页 说明书10页 附图3页

(54)发明名称

一种数据库死锁的处理方法、装置和数据库系统

(57)摘要

本申请公开一种数据库死锁的处理方法、装置和数据库系统，该方法包括：将事务划分成顺序执行的N个子事务；在该N个子事务中的每一个子事务执行之前创建保存点并记录；顺序执行该N个子事务；当该N个子事务中的第n个子事务发生锁等待且等待时间大于该第n个子事务对应的第一等待时间时，将该事务回滚到回滚保存点，该回滚保存点为执行该第n个子事务之前创建的一个保存点；继续顺序执行该N个子事务中在该回滚保存点之后的子事务。本申请通过将事务拆分成多个子事务并在执行前设置保存点，在其中的子事务发生锁等待超过该子事务对应的第一等待时间后回滚到该子事务之前的保存点并继续执行，从而能够在一定程度提高发生死锁的事务的执行效率。




CN 108205464 A

1.一种数据库死锁的处理方法,其特征在于,包括:

将事务划分成顺序执行的N个子事务,其中,N为大于1的正整数;

在所述N个子事务中的每一个子事务执行之前创建保存点;

顺序执行所述N个子事务;

当所述N个子事务中的第n个子事务发生锁等待且等待时间大于所述第n个子事务对应的第一等待时间时,将所述事务回滚到回滚保存点,其中,n为小于或等于N的正整数,所述回滚保存点为执行所述第n个子事务之前创建的一个保存点;

继续顺序执行所述N个子事务中在所述回滚保存点之后的子事务。

2.如权利要求1所述的方法,其特征在于,所述方法还包括:

当所述N个子事务中的第n个子事务第i次发生锁等待且等待时间大于所述第n个子事务对应的第一等待时间,根据第i次锁等待对应的需要回滚的子事务个数s,确定所述回滚保存点,s与i的映射关系是预设的,i、s都为正整数。

3.如权利要求2所述的方法,其特征在于,所述根据第i次锁等待对应的需要回滚的子事务个数s,确定所述回滚保存点包括:

当s＜n时,确定第n-s+1个子事务之前的第一个保存点为所述回滚保存点;或者

当s≥n时,确定第1个子事务之前的保存点为所述回滚保存点。

4.如权利要求1-3任一项所述的方法,其特征在于,

所述第一等待时间是所述事务所在的数据库系统的用户设置的;或者

所述第一等待时间是按照随机数设置的。

5.如权利要求1-4中任一项所述的方法,其特征在于,所述继续顺序执行所述N个子事务中在所述回滚保存点之后的子事务包括:在回滚到所述回滚保存点且等待第二等待时间之后,继续顺序执行所述N个子事务中在所述回滚保存点之后的子事务。

6.如权利要求5所述的方法,其特征在于,

所述第二等待时间是根据所述事务所涉及的结构化查询语言SQL类型设定的;或者

所述第二等待时间是所述事务所在的数据库系统的用户设置的;或者

所述第二等待时间按照随机数设置的。

7.如权利要求1-6中任一项所述的方法,其特征在于,所述方法还包括:在创建所述N个子事务中的每一个子事务对应的保存点时,按顺序记录所述保存点,并在将所述事务回滚到所述回滚保存点时,删除所述回滚保存点之后创建的保存点的记录。

8.如权利要求7所述的方法,其特征在于,

所述N个子事务中的每一个子事务在执行之前创建的保存点按先后顺序存储于链表中,所述回滚保存点是通过查找所述链表确定的。

9.如权利要求1-8中任一项所述的方法,其特征在于,

所述N个子事务中的每一个子事务在执行之前创建的保存点是根据保存点的序号和预定规则命名的,所述回滚保存点是根据所述回滚保存点的序号和所述预定规则确定的。

10.一种处理数据库死锁的装置,其特征在于,包括:事务划分单元、执行单元、保存点创建单元和事务回滚单元,其中,

所述事务划分单元将事务划分成顺序执行的N个子事务,其中,N为大于1的正整数;

所述执行单元顺序执行所述N个子事务;

所述保存点创建单元在执行所述N个子事务中的每一个子事务之前创建保存点；

当所述N个子事务中的第n个子事务发生锁等待且等待时间大于所述第n个子事务对应的第一等待时间时，所述事务回滚单元将所述事务回滚到回滚保存点，其中，n为小于等于N的正整数，所述回滚保存点为执行所述第n个子事务之前创建的一个保存点；

所述执行单元还在所述事务回滚单元将所述事务回滚到所述回滚保存点之后，继续顺序执行所述N个子事务中在所述回滚保存点之后的子事务。

11.如权利要求10所述的装置，其特征在于，所述装置还包括确定单元，当所述N个子事务中的第n个子事务第i次发生锁等待且等待时间大于所述第n个子事务对应的第一等待时间，所述确定单元根据第i次锁等待对应的需要回滚的子事务个数s，确定所述回滚保存点，s与i的映射关系是预设的，i、s都为正整数。

12.如权利要求11所述的装置，其特征在于，所述确定单元根据第i次锁等待对应的需要回滚的子事务个数s，确定所述回滚保存点包括：

所述确定单元确定第n-s+1个子事务之前的第一个保存点为所述回滚保存点，其中，s＜n；或者

所述确定单元确定第1个子事务之前的保存点为所述回滚保存点，其中，s≥n。

13.如权利要求10-12任一项所述的装置，其特征在于，

所述第一等待时间是所述事务所在的数据库系统的用户设置的；或者

所述第一等待时间是按照随机数设置的。

14.如权利要求10-13中任一项所述的装置，其特征在于，所述执行单元具体在回滚到所述回滚保存点且等待第二等待时间之后，继续顺序执行所述N个子事务中在所述回滚保存点之后的子事务。

15.如权利要求14所述的装置，其特征在于，

所述第二等待时间是根据所述事务所涉及的结构化查询语言SQL类型设定的；或者

所述第二等待时间是所述事务所在的数据库系统的用户设置的；或者

所述第二等待时间按照随机数设置的。

16.如权利要求10-15中任一项所述的装置，其特征在于，所述装置还包括删除单元，其中，

所述保存点创建单元还在创建所述N个子事务中的每一个子事务对应的保存点时，按顺序记录所述保存点；

所述删除单元还在所述事务回滚单元将所述事务回滚到所述回滚保存点时，删除所述回滚保存点之后创建的保存点的记录。

17.如权利要求16所述的装置，其特征在于，

所述N个子事务中的每一个子事务在执行之前创建的保存点按先后顺序存储于链表中，所述回滚保存点是通过查找所述链表确定的。

18.如权利要求10-17中任一项所述的装置，其特征在于，

所述N个子事务中的每一个子事务在执行之前创建的保存点是根据保存点的序号和预定规则命名的，所述回滚保存点是根据所述回滚保存点的序号和所述预定规则确定的。

# 一种数据库死锁的处理方法、装置和数据库系统

## 技术领域

[0001] 本申请涉及数据库领域，尤其涉及数据库死锁的处理方法、装置和数据库系统。

## 背景技术

[0002] 数据库死锁是数据库中常见的问题。产生死锁一般需要以下四个必要条件：

[0003] (1)互斥条件：一个资源每次只能被一个进程使用。

[0004] (2)请求与保持条件：一个进程因请求资源而阻塞时，对已获得的资源保持不放。

[0005] (3)不剥夺条件:进程已获得的资源，在末使用完之前，不能强行剥夺。

[0006] (4)循环等待条件:若干进程之间形成一种头尾相接的循环等待资源关系。

[0007] 现有数据库中解决死锁的方法，通常是由数据库来判定是否出现死锁，如果发现死锁则回滚整个事务操作，由应用程序进行重试或需要人为干预。如果事务需要很长时间才能执行完，那么当事务因执行到中间遇到死锁而被迫回滚时，事务需要从头开始执行，会耗费大量不必要的时间。

[0008] 如何提高发生死锁的事务的执行效率，是本申请实施例所要解决的技术问题。

## 发明内容

[0009] 本申请实施例的目的是提供一种数据库死锁的处理方法、装置和数据库系统，以提高发生死锁的事务的执行效率。

[0010] 为解决上述技术问题，本申请实施例是这样实现的：

[0011] 第一方面，提供了一种数据库死锁的处理方法，该方法包括：将事务划分成顺序执行的N个子事务，其中，N为大于1的正整数；在该N个子事务中的每一个子事务执行之前创建保存点；顺序执行该N个子事务；当该N个子事务中的第n个子事务发生锁等待且等待时间大于该第n个子事务对应的第一等待时间时，将该事务回滚到回滚保存点，其中，n为小于或等于N的正整数，该回滚保存点为执行该第n个子事务之前创建的一个保存点；继续顺序执行该N个子事务中在该回滚保存点之后的子事务。

[0012] 第二方面，提供了一种处理数据库死锁的装置，该装置包括：事务划分单元、执行单元、保存点创建单元和事务回滚单元，其中，该事务划分单元将事务划分成顺序执行的N个子事务，其中，N为大于1的正整数；该执行单元顺序执行该N个子事务；该保存点创建单元在执行该N个子事务中的每一个子事务之前创建保存点；当该N个子事务中的第n个子事务发生锁等待且等待时间大于该第n个子事务对应的第一等待时间时，该事务回滚单元将该事务回滚到回滚保存点，其中，n为小于等于N的正整数，该回滚保存点为执行该第n个子事务之前创建的一个保存点；该执行单元还在该事务回滚单元将该事务回滚到该回滚保存点之后，继续顺序执行该N个子事务中在该回滚保存点之后的子事务。

[0013] 本申请实施例采用的上述至少一个技术方案能够达到以下有益效果：通过将事务拆分成多个子事务并在执行前设置保存点，在其中的子事务发生锁等待超过该子事务对应的第一等待时间后回滚到该子事务之前的保存点并继续执行，从而能够在一定程度提高发

生死锁的事务的执行效率。

## 附图说明

[0014] 此处所说明的附图用来提供对本申请的进一步理解，构成本申请的一部分，本申请的示意性实施例及其说明用于解释本申请，并不构成对本申请的不当限定。在附图中：

[0015] 图1为本申请的一个实施例数据库死锁的处理方法流程图。

[0016] 图2是本申请的一个实施例数据库死锁的具体处理方法流程图。

[0017] 图3是本申请的一个实施例电子设备的结构示意图。

[0018] 图4是本申请的一个实施例处理数据库死锁的装置的结构示意图。

[0019] 图5是本申请的另一个实施例处理数据库死锁的装置的结构示意图。

## 具体实施方式

[0020] 为使本申请的目的、技术方案和优点更加清楚，下面将结合本申请具体实施例及相应的附图对本申请技术方案进行清楚、完整地描述。显然，所描述的实施例仅是本申请一部分实施例，而不是全部的实施例。基于本申请中的实施例，本领域普通技术人员在没有做出创造性劳动前提下所获得的所有其他实施例，都属于本申请保护的范围。

[0021] 以下结合附图，详细说明本申请各实施例提供的技术方案。

[0022] 图1是本申请的一个实施例数据库死锁的处理方法流程图。图1的方法由处理数据库死锁的装置执行。在实际的应用中，该处理数据库死锁的装置可以是数据库系统的一个功能模块。该方法包括：

[0023] S110，将事务划分成顺序执行的N个子事务。

[0024] 其中，N为大于1的正整数。

[0025] 事务(Transaction)是访问并可能更新数据库中各种数据项的一个程序执行单元(unit)。事务由事务开始(begin transaction)和事务结束(end transaction)之间执行的全体操作组成。例如：在关系数据库中，一个事务可以是一条结构化查询语言(Structured Query Language，SQL)语句，一组SQL语句或整个程序。

[0026] 应理解，在本申请中，该事务包含多条插入(Insert)、更新(Update)或删除(Delete)的SQL语句等涉及数据库修改的数据库操纵语句，能够进一步划分为多个子事务。

[0027] 例如，假设该事务由以下SQL语句组成：

[0028] delete from table1where col1＝”value1”；

[0029] insert into table2values(value1,value2,value3……)；

[0030] update table3set col2＝”value4”where col3＝“value5”。

[0031] 该事务可以进一步划分为多个子事务。

[0032] 例如，该事务可以分为子事务1和子事务2，其中，

[0033] 子事务1为：

[0034] delete from table1where col1＝”value1”；

[0035] insert into table2values(value1,value2,value3……)；

[0036] 子事务2为：

[0037] update table3set col2＝”value4”where col3＝“value5”。

[0038] 又例如,该事务还可以分为子事务1、子事务2和子事务3,其中,

[0039] 子事务1为:

[0040] delete from table1where col1＝”value1”;

[0041] 子事务2为:

[0042] insert into table2values(value1,value2,value3……);

[0043] 子事务3为:

[0044] update table3set col2＝”value4”where col3＝“value5”。

[0045] 在具体的应用中,可以根据其中的SQL语句的多少进行子事务的划分,或者根据需要进行插入(Insert)、更新(Update)或删除(Delete)的记录的数量进行子事务的划分,或者根据预先估计的执行时间进行子事务的划分,等待,本申请在此不做限制。

[0046] S120,在该N个子事务中的每一个子事务执行之前创建保存点。

[0047] 保存点(savepoint)是事务过程中的一个逻辑点,通过保存点可以把事务回退到保存点的状态,而不必回退整个事务。

[0048] 创建保存点的方式可参考现有技术。例如,在Oracle数据库中,可通过savepoint savepoint_name这条命令创建保存点。其中,savepoint_name表示保存点名称。

[0049] S130,顺序执行该N个子事务。

[0050] S140,当该N个子事务中的第n个子事务发生锁等待且等待时间大于该第n个子事务对应的第一等待时间时,将该待处理事务回滚到回滚保存点。

[0051] 其中,n为小于或等于N的正整数,该回滚保存点为执行该第n个子事务之前创建的一个保存点。

[0052] 应理解,本申请的第n个子事务,是指该N个子事务中顺序执行的第n个子事务。

[0053] S150,继续顺序执行该N个子事务中在该回滚保存点之后的子事务。

[0054] 在本申请实施例中,通过将事务拆分成多个子事务并在执行前设置保存点,在其中的子事务发生锁等待超过该子事务对应的第一等待时间后回滚到该子事务之前的保存点并继续执行,从而能够在一定程度提高发生死锁的事务的执行效率。

[0055] 在步骤S140中,该回滚保存点是在执行第n个子事务之前创建的保存点。该回滚保存点的选择决定了该事务需要回滚的子事务的数量。因此,在步骤S140之前,该方法还可包括:确定该回滚保存点。

[0056] 可选地,作为一个实施例,确定该回滚保存点具体可实现为:当该N个子事务中的第n个子事务第i次发生锁等待且等待时间大于该第n个子事务对应的第一等待时间,根据第i次锁等待对应的需要回滚的子事务个数s,确定该回滚保存点,s与i的映射关系是预设的,i、s都为正整数。

[0057] 应理解,s与i的映射关系是预设的,例如可以预先设定s与i的映射关系表,或者,预先设定s与i之间的函数关系,等等。

[0058] 具体地,例如,s与i的映射关系表可如表1所示:

[0059] 表1:

[0060]

| 锁等待次数 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 需要回滚的子事务个数 | 1 | 2 | 3 | 5 | 8 |

[0061] 或者，例如，可规定s与i的函数关系为s＝i。

[0062] 当然，应理解，上述列举的映射关系表和函数关系仅仅是示意性的，在实际的应用中，可根据实际情况进行设置。

[0063] 此外，应理解，由于上述需要回滚的子事务个数s可能大于当前已经执行的子事务的个数n，因此，当s＜n时，可确定需要回滚s个子事务；当s≥n，可确定需要回滚n个子事务。换句话说，当s＜n时，确定第n-s+1个子事务之前的第一个保存点为该回滚保存点；或者，当s≥n时，确定第1个子事务之前的保存点为该回滚保存点。

[0064] 在本实现方式中，通过根据子事务发生锁等待且等待时间大于第一等待时间的次数，确定需要回滚的子事务的个数，进而确定回滚的保存点，从而能够避免因回滚的子事务设置过少而无法释放锁，或者因回滚的子事务设置过多而导致不必要的事务回滚。

[0065] 当然，应理解，上述实现方式只是示出了一种可能的实现方式，在具体的应用中，还可通过其它方式确定回滚保存点。

[0066] 应理解，该N个子事务中每个子事务发生锁等待对应的第一等待时间可以相同，也可以不同。

[0067] 可选地，该第一等待时间可以是数据库的用户设置的。用户可为第一等待时间设定一个固定值，或者，或者根据某种配置规则设定该第一等待时间。根据参数设置第一等待时间，例如，根据第n个子事务的涉及的数据操作类型、所述第n个子事务涉及的数据操作的记录数、所述第n个子事务的序号n以及所述第n个子事务发生锁等待的次数中的至少一种参数，设置第一等待时间。

[0068] 或者，可选地，作为另一个实施例，该第一等待时间是根据随机数设置的。通过随机数设置第一等待时间，可以防止所有连接超时都是一样的。

[0069] 在本申请实施例中，通过灵活设置子事务发生锁等待的第一等待时间，可以无需用户干预死锁进程，提高数据库的执行效率。

[0070] 在本申请实施例中，通过灵活设置第n个子事务发生锁等待的第一等待时间，能够在使得子事务回滚后释放锁的前提下，一定程度上减少回滚的子事务的数量。

[0071] 可选地，作为一个实施例，步骤S150具体可实现为：在回滚到该回滚保存点且等待第二等待时间之后，继续顺序执行该N个子事务中在该回滚保存点之后的子事务。

[0072] 可选地，该第二等待时间可以是数据库的用户设置的。用户可为第二等待时间设定一个固定值，或者根据某种配置规则设定该第二等待时间。

[0073] 或者，可选地，第二等待时间是根据该事务所涉及的SQL类型确定的。例如，数据定义语言(Data Definition Language，DDL)类型的第二等待时间可以设置长一些；数据操纵语言(data manipulation language，DML)的第二等待时间可以设置短一些，等等。

[0074] 或者，可选地，该第二等待时间是根据随机数设置的。通过随机数设置第二等待时间，可以防止所有连接超时都是一样的。

[0075] 在本申请实施例中，通过在回滚到该回滚保存点后等待第二等待时间，能够在一定程度上避免持有第n个子事务的锁的事务未释放该锁而导致该第n个子事务继续发生锁等待，进一步提高发生死锁的事务的执行效率。

[0076] 可选地，作为一个实施例，该方法还包括：在创建该N个子事务中的每一个子事务对应的保存点时，按顺序记录该保存点；在将该事务回滚到该回滚保存点时，删除该回滚保

存点之后创建的保存点的记录。

[0077] 删除保存点的操作可参考现有技术，例如，在Oracle数据库中，可以通过release savepoint命令删除创建的保存点。具体的命令格式如下

[0078] release savepoint savepoint_name;

[0079] 其中，savepoint_name表示创建的保存点的名称。

[0080] 此外，如果保存点的记录存储于某个链表或其它数据结构中，则还需要从该链表或数据结构中删除该回滚保存点之后创建的保存点的记录。

[0081] 在本申请实施例中，通过删除回滚保存点之后的保存点的记录，使得处理数据库死锁的装置在后续的确定回滚的保存点的操作中能够迅速定位回滚的保存点。

[0082] 当该保存点需要另外进行记录时，可选地，该N个子事务中的每一个子事务在执行之前创建的保存点可按先后顺序存储于链表中，其中，该回滚保存点是通过查找该链表确定的。

[0083] 在本申请实施例中，通过链表查找需要回滚的回滚保存点，能够提高查找回滚保存点的效率，从而能够进一步提高发生死锁的事务的执行效率。

[0084] 或者，可选地，作为另一个实施例，该N个子事务中的每一个子事务在执行之前创建的保存点是按照保存点的序号和预定规则命名的，该回滚保存点是根据该回滚保存点的序号和该预定规则确定的。

[0085] 应理解，保存点的记录不是必须的。例如，可以约定保存点的命名规则为标识名+序号，则当确定回滚保存点的序号后，即可确定回滚保存点的名称，从而找到该保存点。

[0086] 当然，应理解，在实际的应用中，可能只记录保存点的信息，而不会记录回滚的子事务的个数。但是，由于每一个子事务之前都有一个对应的保存点，确定需要回滚的子事务，也就等价于确定回滚的保存点。

[0087] 下面，将结合具体的实施例，对本申请图1所示的实施例做进一步的描述。

[0088] 图2是本申请的一个实施例数据库死锁的具体处理方法流程图。在图2所示的场景中，事务可分成子事务1、子事务2和子事务3共3个子事务，其执行顺序按照子事务1、子事务2和子事务3顺序执行。如图2所示，在执行子事务1之前，建立保存点savepoint1；在执行子事务2之前，建立保存点savepoint2；在执行子事务2之前，建立保存点savepoint3。

[0089] 图2所示实施例中，假设事务执行到子事务3时，发生锁等待。

[0090] 如果执行子事务3第1次发生锁等待且超出第一次锁等待对应的等待时间，需要回滚的子事务数s取值为1，该次回滚需要回滚到第3-1+1＝3个子事务之前的第一个保存点，也即回滚到子事务3之前的第一个保存点savepoint3。同时，数据库需要释放savepoint3之后的锁。

[0091] 如果执行子事务3第2次发生锁等待且超出第一次锁等待对应的等待时间，需要回滚的子事务数s取值为2，该次回滚需要回滚到第3-2+1＝2个子事务之前的第一个保存点，也即回滚到子事务2之前的第一个保存点savepoint2。同时，数据库需要释放savepoint2之后的锁。此外，如果本申请实施例中使用链表等记录保存点，则还需要删除savepoint2之后的保存点记录。

[0092] 如果执行子事务3第3次发生锁等待且超出第一次锁等待对应的等待时间，则需要回滚的子事务数s取值为3，该次回滚需要回滚到第3-3+1＝1个子事务之前的第一个保存

点，也即回滚到子事务1之前的第一个保存点savepoint1。同时，数据库需要释放savepoint3之后的锁。此外，如果本申请实施例中使用链表等记录保存点，则还需要删除savepoint3之后的保存点记录。

[0093] 当然，应理解，如图2所示，可以通过链表保存每个子事务执行前创建的保存点的标识。图2中的3个子事务在执行之前创建的保存点按先后顺序存储于链表中，分别对应执行计划1、执行计划2和执行计划3。数据库可根据需要回滚的子事务的个数，从链表中找到对应的保存点的标识。

[0094] 此外，应理解，在图2所示的场景中，也可以不用链表记录保存点的信息，而是规定保存点的命名规则。例如，可以规定图2中保存点的命名规则为“savepoint”+序号。在第一次回滚时，需要回滚的保存点的序号为3-1+1＝3，根据该命名规则即可确定该保存点为savepoint3，等等。

[0095] 另外，对于用于判断是否超时的锁等待的等待时间，可以规定每次锁等待的等待时间相同，或者根据锁等待发生超时的次数、锁等待发生时的子事务的序号、锁等待发生时的子事务涉及的数据操作类型、或者是锁等待发生时的子事务涉及的数据操作的记录数等因素，综合确定锁等待的等待时间。此外，也可以根据随机数，随机分配所等待的等待时间。

[0096] 此外，每次发生回滚后，还可设定一个等待时间，当回滚后经过该等待时间后，再继续执行子事务，可避免持有本事务的锁的事务未能及时释放掉该锁而导致本事务继续发生锁等待。

[0097] 需要说明的是，上述如图1所示实施例所提供方法的各步骤的执行主体均可以是同一设备，或者，该方法也由不同设备作为执行主体。比如，步骤S110和步骤S120的执行主体可以为设备1，步骤S130的执行主体可以为设备2；又比如，步骤S110的执行主体可以为设备1，步骤S120和步骤S130的执行主体可以为设备2；等等。

[0098] 图3是本本申请的一个实施例电子设备的结构示意图。请参考图3，在硬件层面，该电子设备包括处理器、内部总线、网络接口、内存以及非易失性存储器，当然还可能包括其他业务所需要的硬件。处理器从非易失性存储器中读取对应的计算机程序到内存中然后运行，在逻辑层面上形成用户界面解锁的装置。当然，除了软件实现方式之外，本申请并不排除其他实现方式，比如逻辑器件抑或软硬件结合的方式等等，也就是说以下处理流程的执行主体并不限定于各个逻辑单元，也可以是硬件或逻辑器件。

[0099] 图4是本申请的一个实施例的处理数据库死锁的装置400的结构示意图。应理解，处理数据库死锁的装置400可以是数据库系统，或者是数据库系统中的一个具体的实现模块，本申请实施例在此不作限制。请参考图4，在软件实施方式中，该数据库死锁的装置400可包括：事务划分单元410、保存点创建单元420、执行单元430和事务回滚单元440。其中，

[0100] 事务划分单元410将事务划分成顺序执行的N个子事务，其中，N为大于1的正整数；

[0101] 保存点创建单元420在执行该N个子事务中的每一个子事务之前创建保存点；

[0102] 执行单元430顺序执行该N个子事务；

[0103] 当该N个子事务中的第n个子事务发生锁等待且等待时间大于该第n个子事务对应的第一等待时间时，事务回滚单元440将该事务回滚到回滚保存点，其中，n为小于等于N的正整数，该回滚保存点为执行该第n个子事务之前创建的一个保存点；

[0104] 执行单元430还在该事务回滚单元440将该待执行事务回滚到该第一保存点之后，

继续顺序执行该N个子事务中在该第一保存点之后的子事务。

[0105] 在本申请实施例中，通过将事务拆分成多个子事务并在执行前设置保存点，在其中的子事务发生锁等待超过该子事务对应的第一等待时间后回滚到该子事务之前的保存点并继续执行，从而能够在一定程度提高发生死锁的事务的执行效率。

[0106] 图5是本申请的另一个实施例的处理数据库死锁的装置400的结构示意图。可选地，如图5所示，装置400还可包括确定单元450，当该N个子事务中的第n个子事务第i次发生锁等待且等待时间大于该第n个子事务对应的第一等待时间，该确定单元450根据第i次锁等待对应的需要回滚的子事务个数s，确定该回滚保存点，s与i的映射关系是预设的，i、s都为正整数。

[0107] 进一步地，确定单元450根据第i次锁等待对应的需要回滚的子事务个数s，确定该回滚保存点包括：确定单元450可确定第n-s个子事务之前的第一个保存点为该第一保存点，其中，s＜n；或者，确定单元45可确定第1个子事务之前的保存点为该第一保存点，其中，s≥n。

[0108] 可选地，该第一等待时间是该事务所在的数据库系统的用户设置的；或者，该第一等待时间是按照随机数设置的。

[0109] 可选地，执行单元430具体在回滚到该第一保存点且等待第二等待时间之后，继续顺序执行该N个子事务中在该第一保存点之后的子事务。进一步地，该第二等待时间是根据该事务所涉及的结构化查询语言SQL类型设定的；或者，该第二等待时间是该事务所在的数据库系统的用户设置的；或者，该第二等待时间按照随机数设置的。

[0110] 可选地，作为一个实施例，该N个子事务中的每一个子事务在执行之前创建的保存点是根据保存点的序号和预定规则命名的，该回滚保存点是根据该回滚保存点的序号和该预定规则确定的。

[0111] 或者，可选地，作为另一个实施例，该N个子事务中的每一个子事务在执行之前创建的保存点按先后顺序存储于链表中，该回滚保存点是通过查找该链表确定的。进一步地，如图5所示，装置400还可包括删除单元460。如果保存点创建单元在创建该N个子事务中的每一个子事务对应的保存点时，还按顺序记录该保存点；则该删除单元可在该事务回滚单元将该事务回滚到该回滚保存点时，删除该回滚保存点之后创建的保存点的记录。

[0112] 处理数据库死锁的装置400还可执行图1的方法，并实现数据库或处理数据库死锁的装置在图1、图2所示实施例的功能，本申请实施例在此不再赘述。

[0113] 本申请还公开了一种数据库系统，包括图4或图5所示实施例的处理数据库死锁的装置400。

[0114] 在20世纪90年代，对于一个技术的改进可以很明显地区分是硬件上的改进(例如，对二极管、晶体管、开关等电路结构的改进)还是软件上的改进(对于方法流程的改进)。然而，随着技术的发展，当今的很多方法流程的改进已经可以视为硬件电路结构的直接改进。设计人员几乎都通过将改进的方法流程编程到硬件电路中来得到相应的硬件电路结构。因此，不能说一个方法流程的改进就不能用硬件实体模块来实现。例如，可编程逻辑器件(Programmable Logic Device,PLD)(例如现场可编程门阵列(Field Programmable Gate Array,FPGA))就是这样一种集成电路，其逻辑功能由用户对器件编程来确定。由设计人员自行编程来把一个数字系统“集成”在一片PLD上，而不需要请芯片制造厂商来设计和制作

专用的集成电路芯片。而且，如今，取代手工地制作集成电路芯片，这种编程也多半改用“逻辑编译器(logic compiler)”软件来实现，它与程序开发撰写时所用的软件编译器相类似，而要编译之前的原始代码也得用特定的编程语言来撰写，此称之为硬件描述语言(Hardware Description Language，HDL)，而HDL也并非仅有一种，而是有许多种，如ABEL(Advanced Boolean Expression Language)、AHDL(Altera Hardware Description Language)、Confluence、CUPL(Cornell University Programming Language)、HDCal、JHDL(Java Hardware Description Language)、Lava、Lola、MyHDL、PALASM、RHDL(Ruby Hardware Description Language)等，目前最普遍使用的是VHDL(Very-High-Speed Integrated Circuit Hardware Description Language)与Verilog。本领域技术人员也应该清楚，只需要将方法流程用上述几种硬件描述语言稍作逻辑编程并编程到集成电路中，就可以很容易得到实现该逻辑方法流程的硬件电路。

[0115] 控制器可以按任何适当的方式实现，例如，控制器可以采取例如微处理器或处理器以及存储可由该(微)处理器执行的计算机可读程序代码(例如软件或固件)的计算机可读介质、逻辑门、开关、专用集成电路(Application Specific Integrated Circuit，ASIC)、可编程逻辑控制器和嵌入微控制器的形式，控制器的例子包括但不限于以下微控制器：ARC 625D、Atmel AT91SAM、Microchip PIC18F26K20以及Silicone Labs C8051F320，存储器控制器还可以被实现为存储器的控制逻辑的一部分。本领域技术人员也知道，除了以纯计算机可读程序代码方式实现控制器以外，完全可以通过将方法步骤进行逻辑编程来使得控制器以逻辑门、开关、专用集成电路、可编程逻辑控制器和嵌入微控制器等的形式来实现相同功能。因此这种控制器可以被认为是一种硬件部件，而对其内包括的用于实现各种功能的装置也可以视为硬件部件内的结构。或者甚至，可以将用于实现各种功能的装置视为既可以是实现方法的软件模块又可以是硬件部件内的结构。

[0116] 上述实施例阐明的系统、装置、模块或单元，具体可以由计算机芯片或实体实现，或者由具有某种功能的产品来实现。一种典型的实现设备为计算机。具体的，计算机例如可以为个人计算机、膝上型计算机、蜂窝电话、相机电话、智能电话、个人数字助理、媒体播放器、导航设备、电子邮件设备、游戏控制台、平板计算机、可穿戴设备或者这些设备中的任何设备的组合。

[0117] 为了描述的方便，描述以上装置时以功能分为各种单元分别描述。当然，在实施本申请时可以把各单元的功能在同一个或多个软件和/或硬件中实现。

[0118] 本领域内的技术人员应明白，本申请的实施例可提供为方法、系统、或计算机程序产品。因此，本申请可采用完全硬件实施例、完全软件实施例、或结合软件和硬件方面的实施例的形式。而且，本申请可采用在一个或多个其中包含有计算机可用程序代码的计算机可用存储介质(包括但不限于磁盘存储器、CD-ROM、光学存储器等)上实施的计算机程序产品的形式。

[0119] 本申请是参照根据本申请实施例的方法、设备(系统)、和计算机程序产品的流程图和/或方框图来描述的。应理解可由计算机程序指令实现流程图和/或方框图中的每一流程和/或方框、以及流程图和/或方框图中的流程和/或方框的结合。可提供这些计算机程序指令到通用计算机、专用计算机、嵌入式处理机或其他可编程数据处理设备的处理器以产生一个机器，使得通过计算机或其他可编程数据处理设备的处理器执行的指令产生用于实

现在流程图一个流程或多个流程和/或方框图一个方框或多个方框中指定的功能的装置。

[0120] 这些计算机程序指令也可存储在能引导计算机或其他可编程数据处理设备以特定方式工作的计算机可读存储器中，使得存储在该计算机可读存储器中的指令产生包括指令装置的制造品，该指令装置实现在流程图一个流程或多个流程和/或方框图一个方框或多个方框中指定的功能。

[0121] 这些计算机程序指令也可装载到计算机或其他可编程数据处理设备上，使得在计算机或其他可编程设备上执行一系列操作步骤以产生计算机实现的处理，从而在计算机或其他可编程设备上执行的指令提供用于实现在流程图一个流程或多个流程和/或方框图一个方框或多个方框中指定的功能的步骤。

[0122] 在一个典型的配置中，计算设备包括一个或多个处理器(CPU)、输入/输出接口、网络接口和内存。

[0123] 内存可能包括计算机可读介质中的非永久性存储器，随机存取存储器(RAM)和/或非易失性内存等形式，如只读存储器(ROM)或闪存(flash RAM)。内存是计算机可读介质的示例。

[0124] 计算机可读介质包括永久性和非永久性、可移动和非可移动媒体可以由任何方法或技术来实现信息存储。信息可以是计算机可读指令、数据结构、程序的模块或其他数据。计算机的存储介质的例子包括，但不限于相变内存(PRAM)、静态随机存取存储器(SRAM)、动态随机存取存储器(DRAM)、其他类型的随机存取存储器(RAM)、只读存储器(ROM)、电可擦除可编程只读存储器(EEPROM)、快闪记忆体或其他内存技术、只读光盘只读存储器(CD-ROM)、数字多功能光盘(DVD)或其他光学存储、磁盒式磁带，磁带磁磁盘存储或其他磁性存储设备或任何其他非传输介质，可用于存储可以被计算设备访问的信息。按照本文中的界定，计算机可读介质不包括暂存电脑可读媒体(transitory media)，如调制的数据信号和载波。

[0125] 还需要说明的是，术语“包括”、“包含”或者其任何其他变体意在涵盖非排他性的包含，从而使得包括一系列要素的过程、方法、商品或者设备不仅包括那些要素，而且还包括没有明确列出的其他要素，或者是还包括为这种过程、方法、商品或者设备所固有的要素。在没有更多限制的情况下，由语句“包括一个……”限定的要素，并不排除在包括所述要素的过程、方法、商品或者设备中还存在另外的相同要素。

[0126] 本领域技术人员应明白，本申请的实施例可提供为方法、系统或计算机程序产品。因此，本申请可采用完全硬件实施例、完全软件实施例或结合软件和硬件方面的实施例的形式。而且，本申请可采用在一个或多个其中包含有计算机可用程序代码的计算机可用存储介质(包括但不限于磁盘存储器、CD-ROM、光学存储器等)上实施的计算机程序产品的形式。

[0127] 本申请可以在由计算机执行的计算机可执行指令的一般上下文中描述，例如程序模块。一般地，程序模块包括执行特定任务或实现特定抽象数据类型的例程、程序、对象、组件、数据结构等等。也可以在分布式计算环境中实践本申请，在这些分布式计算环境中，由通过通信网络而被连接的远程处理设备来执行任务。在分布式计算环境中，程序模块可以位于包括存储设备在内的本地和远程计算机存储介质中。

[0128] 本说明书中的各个实施例均采用递进的方式描述，各个实施例之间相同相似的部分互相参见即可，每个实施例重点说明的都是与其他实施例的不同之处。尤其，对于系统实

施例而言，由于其基本相似于方法实施例，所以描述的比较简单，相关之处参见方法实施例的部分说明即可。

[0129] 以上所述仅为本申请的实施例而已，并不用于限制本申请。对于本领域技术人员来说，本申请可以有各种更改和变化。凡在本申请的精神和原理之内所作的任何修改、等同替换、改进等，均应包含在本申请的权利要求范围之内。

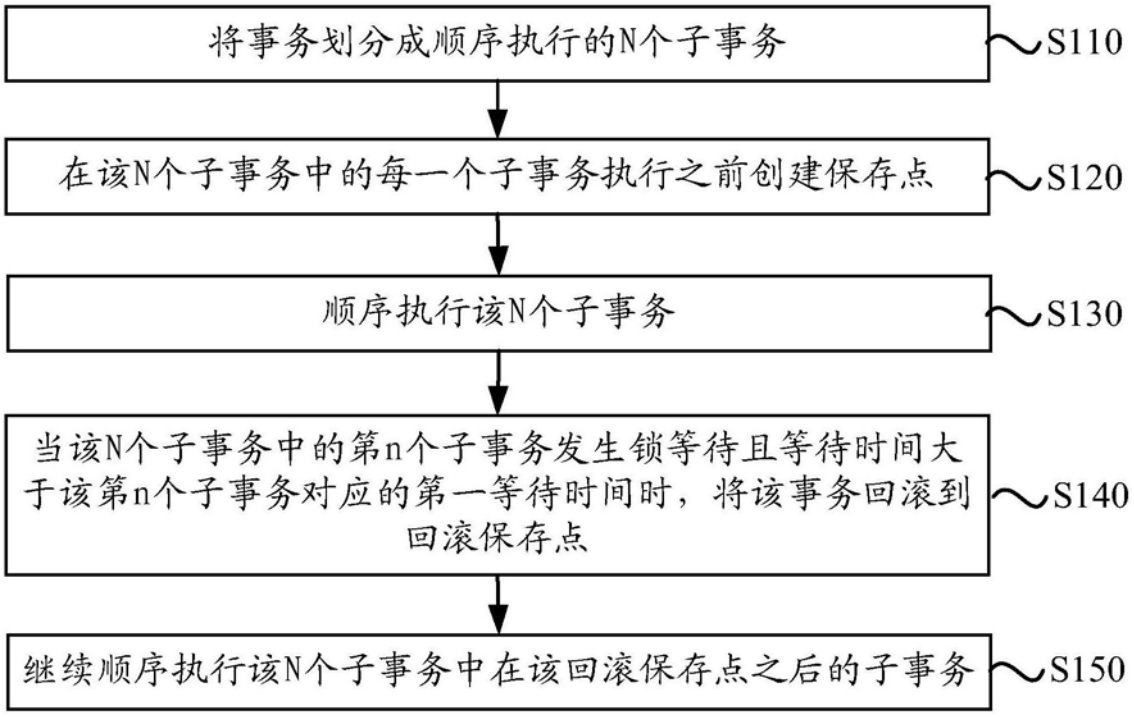

将事务划分成顺序执行的N个子事务
S110
在该N个子事务中的每一个子事务执行之前创建保存点
S120
顺序执行该N个子事务
S130
当该N个子事务中的第n个子事务发生锁等待且等待时间大于该第n个子事务对应的第一等待时间时，将该事务回滚到回滚保存点
S140
继续顺序执行该N个子事务中在该回滚保存点之后的子事务
S150

图1

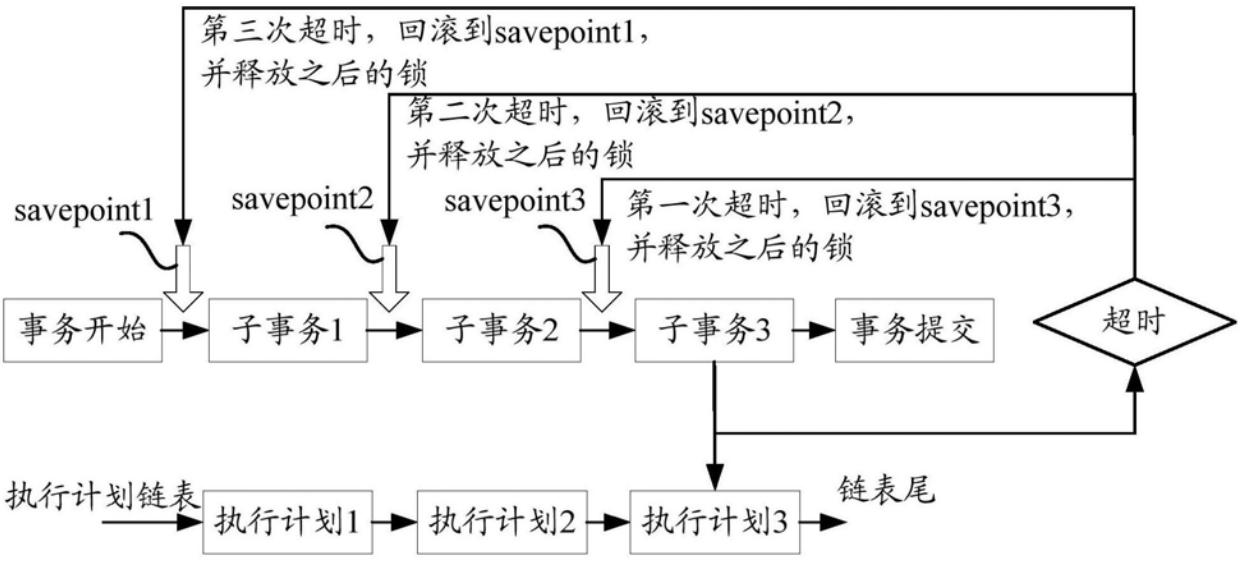

第三次超时，回滚到savepoint1，并释放之后的锁
第二次超时，回滚到savepoint2，并释放之后的锁
第一次超时，回滚到savepoint3，并释放之后的锁
savepoint1
savepoint2
savepoint3
事务开始
子事务1
子事务2
子事务3
事务提交
超时
执行计划链表
执行计划1
执行计划2
执行计划3
链表尾

图2

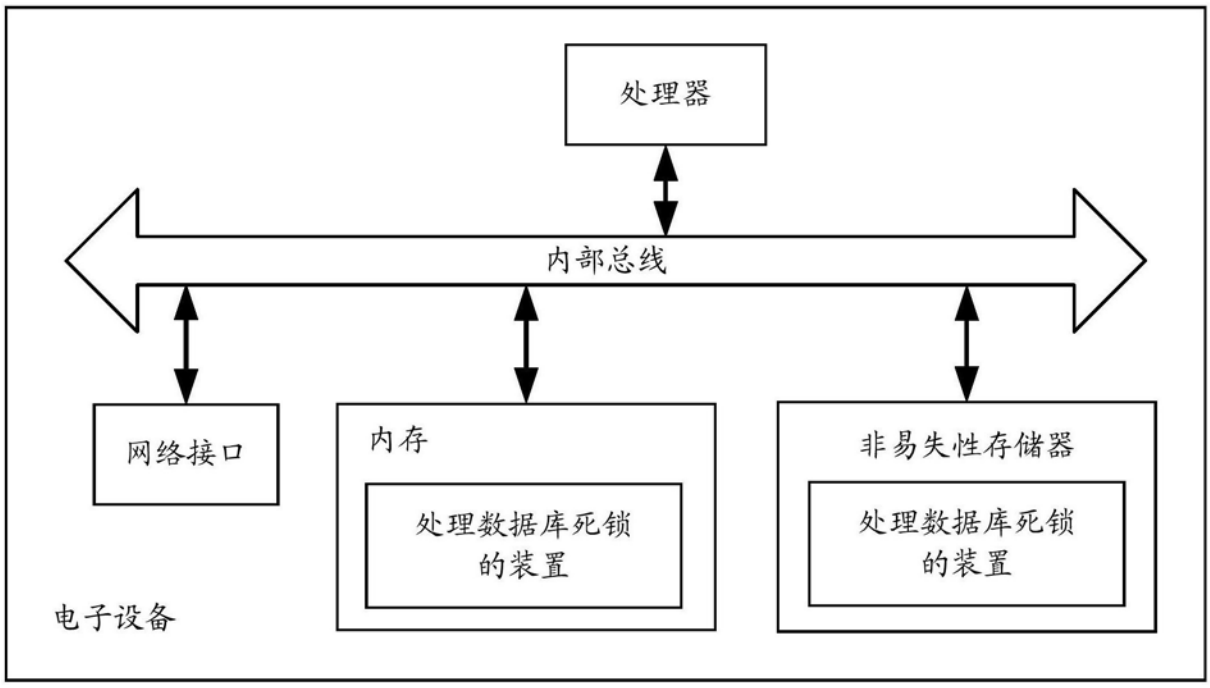

处理器
内部总线
网络接口
内存
处理数据库死锁的装置
非易失性存储器
处理数据库死锁的装置
电子设备

图3

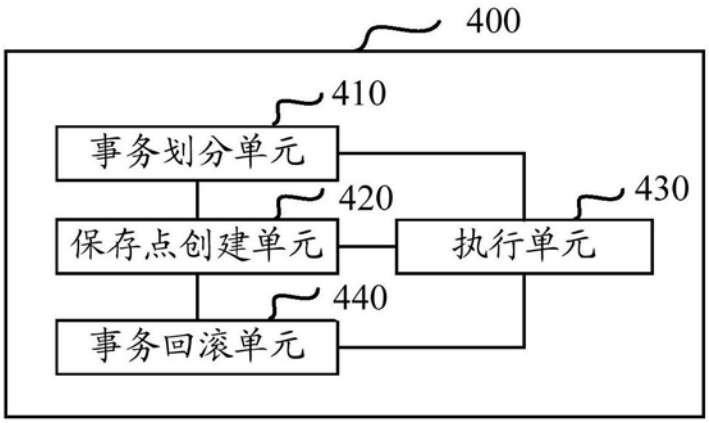

400
410
事务划分单元
420
保存点创建单元
430
执行单元
440
事务回滚单元

图4

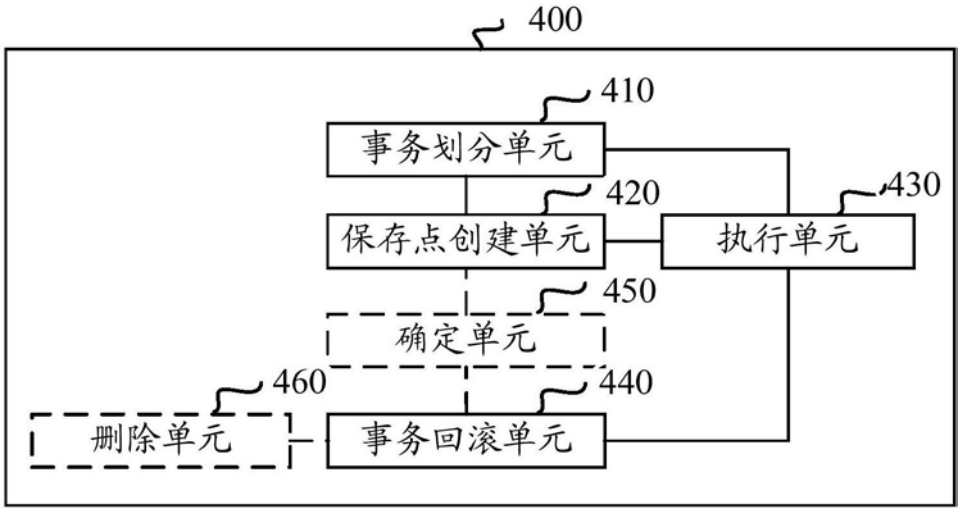

400
410
事务划分单元
420
保存点创建单元
430
执行单元
450
确定单元
460
删除单元
440
事务回滚单元

图5